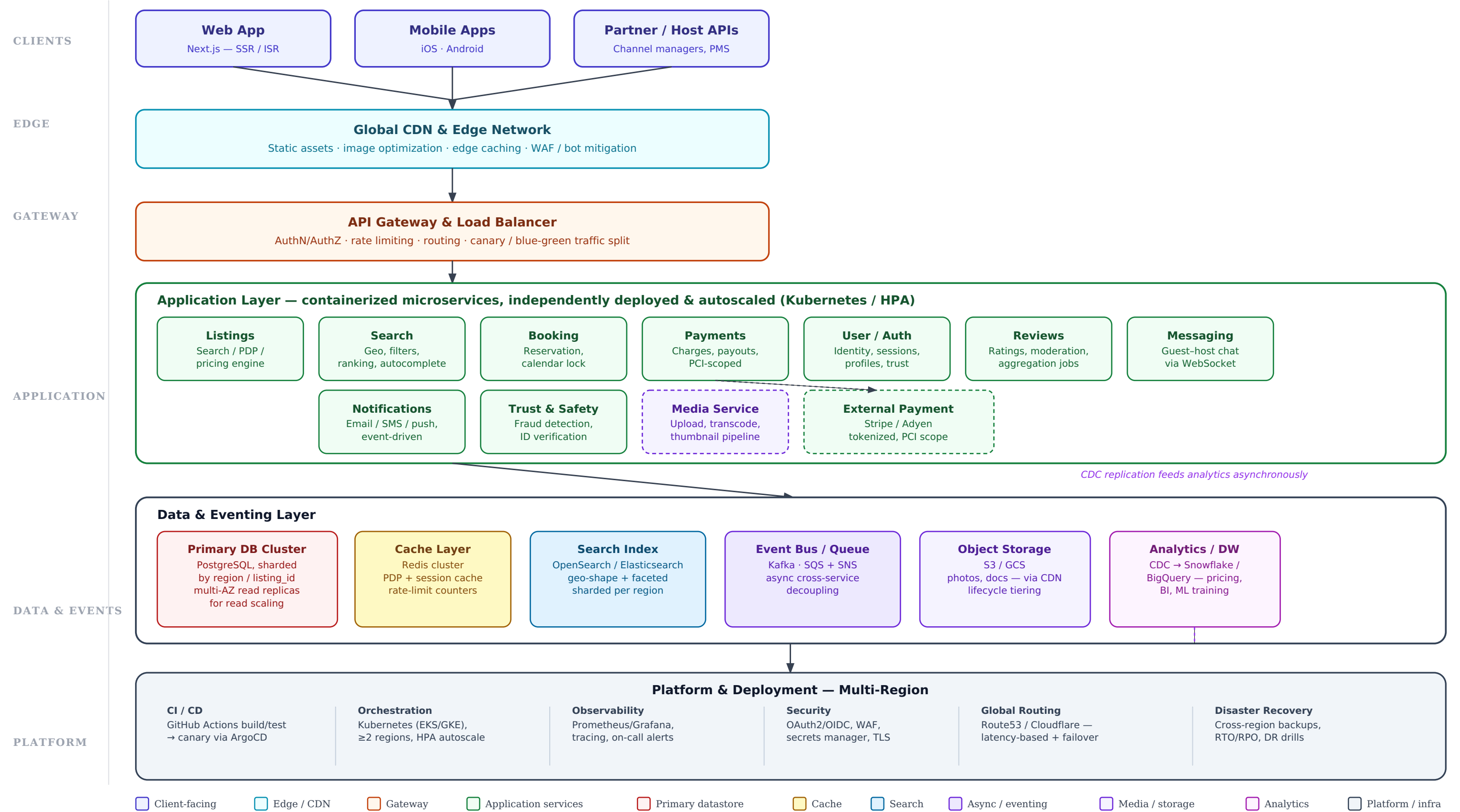


Vacation Rental Marketplace — Production Architecture

High-level scaling strategy: Frontend · Backend · Storage · Search · Deployment

Reference architecture · v1.0
Desktop web + mobile + partner clients



Scaling Strategy — Supporting Notes

How each layer scales as traffic, listing inventory, and geography grow

●Frontend

- Next.js: SSG for stable marketing/SEO pages, ISR for listing detail pages (revalidate on price/availability change), SSR only for personalized/search-result pages.
- Rendered HTML and static assets served through a global CDN with edge caching keyed by locale + currency.
- Images served via an on-the-fly resizing/CDN service to avoid oversized payloads and support responsive images.
- Route-level code-splitting; listing-page path prioritized for Core Web Vitals.
- Feature flags / A-B testing at the edge to ship UI changes gradually without redeploying.

●Backend / Services

- Domain-oriented microservices (Listings, Search, Booking, Payments, User, Reviews, Messaging, Notifications, Trust & Safety, Media) — independently deployable and scaled.
- Stateless instances behind the gateway/load balancer, horizontally autoscaled (HPA) on CPU/RPS via Kubernetes.
- Synchronous calls (REST/gRPC) for read paths; asynchronous events (Kafka/SQS) for cross-service side effects.
- Booking service uses optimistic locking / distributed locks on calendar slots to prevent double-booking.
- Payments isolated into its own PCI-scoped boundary; card data never touches internal services.

●Storage

- PostgreSQL as system of record, sharded by region/listing_id once a single cluster can't absorb write volume; read replicas absorb browse traffic.
- Redis cluster in front of hot paths (listing detail, pricing, sessions) to cut DB load; invalidated on write via events.
- Object storage (S3/GCS) for photos/documents, fronted by CDN, with lifecycle rules tiering cold media to cheaper storage.
- CDC streams DB changes into an analytics warehouse so OLAP workloads never compete with OLTP traffic.

●Search

- Dedicated search index decoupled from the primary DB — listings indexed asynchronously on create/update via the event bus.
- Geo-shape/geo-distance queries power map and radius search; faceted filters handled as index-side aggregations.
- Index sharded and replicated per region for low latency; availability overlaid at query time to avoid stale results.
- Ranking model served from a separate service that re-scores top-N candidates from the index.

●Deployment

- Infrastructure as code (Terraform) across multiple cloud regions for active-active or active-passive redundancy.
- Kubernetes orchestrates containers; CI/CD (GitHub Actions → ArgoCD) performs canary/blue-green rollouts.
- Global DNS/traffic manager routes users to the nearest healthy region with automatic failover.
- Full observability stack (metrics, logs, distributed tracing) with on-call alerting.
- Regular DR drills against defined RTO/RPO; automated cross-region backups.

●Cross-cutting concerns

- Security: OAuth2/OIDC, secrets manager, TLS everywhere, WAF at the edge, least-privilege network policies.
- Rate limiting and fraud detection at the gateway and Trust & Safety service protect booking/payment flows.
- Structured logs + metrics + traces from every service feed a shared observability pipeline.
- Backwards-compatible API versioning so older mobile clients keep working as services evolve.